

Deployment-State Tensor-Arena Verification and Heap Attribution for TinyML on ESP32-S3: A Two-Board Case Study

Aaron George * Ahmed Chyad * Maryam Hafeez* Syed Ali Raza Zaidi*

*School of Electrical and Electronic Engineering, University of Leeds, Leeds, UK

Corresponding author: Aaron George: sc23ag@leeds.ac.uk.

Abstract—Tiny Machine Learning (TinyML) deployment on heterogeneous-memory microcontrollers can be misclassified when arena size or total free memory is measured outside the final firmware state. This paper presents a *deployment-state tensor-arena verification workflow* and heap-attribution study on paired Espressif ESP32-S3 boards running Wi-Fi wireless networking, camera, local Message Queuing Telemetry Transport (MQTT), and cloud/Transport Layer Security (TLS) MQTT workloads. The verifier empirically probes the largest allocatable internal Static Random-Access Memory (SRAM) block and classifies models as internal-safe, borderline, or Pseudo-static Random-Access Memory (PSRAM)-required. Across the deployment ladder, the empirical internal limit fell substantially as camera, messaging, and secure cloud services were enabled. Heap attribution identified `esp_wifi_init()` as the dominant structural allocator, and independent reproduction on a second board strengthens the interpretation that configurable Wi-Fi provisioning is a real placement lever, while the exact threshold remains build-specific. Forced PSRAM placement increased inference latency, showing that placement classification maps to a practical performance cost.

Index Terms—TinyML, ESP32-S3, PSRAM, TensorFlow Lite Micro, tensor arena, heap fragmentation, ESP-IDF, MQTT, embedded inference, deployment verification

I. INTRODUCTION

Tiny Machine Learning (TinyML) systems are evaluated using model size, tensor-arena requirement, inference latency, and total free memory. On the Espressif ESP32-S3, internal Static Random-Access Memory (SRAM) and external Pseudo-static Random-Access Memory (PSRAM) can both be exposed through the capability-aware heap allocator; internal SRAM is preferable for latency, as PSRAM provides larger capacity with slower access and documented access restrictions [6], [8], [12]. The deployment question is therefore not whether a model *fits* in memory, but whether its tensor arena can be placed in internal SRAM in the *final* firmware state.

TensorFlow Lite Micro uses a contiguous tensor arena [1]. A model with arena requirement A_m needs a contiguous allocation of at least A_m bytes. Total free internal memory is insufficient; the critical quantity is the largest allocatable internal block after all deployment components have initialised. Wi-Fi, camera, MQTT, and TLS stacks fragment the heap before inference begins. A model can appear safe in isolation but become borderline or PSRAM-required once the full stack is active.

This paper’s contribution is a **deployment-state placement verification workflow** comprising: (1) empirical contiguous-block probing per firmware state; (2) transition-level heap attribution identifying which initialisation stage causes the dominant collapse; and (3) forced-placement latency benchmarking. The workflow is demonstrated on two boards across five deployment states, including a cloud/Transport Layer Security (TLS) MQTT state. The study intentionally avoids generalising measured byte thresholds beyond the tested Espressif IoT Development Framework (ESP-IDF) version and hardware; the contribution is the methodology and the attribution, not the byte values.

II. BACKGROUND AND RELATED WORK

Tensor arenas and TinyML memory reduction. TensorFlow Lite Micro subdivides a statically allocated arena at runtime [1]. For arena use A_m , internal SRAM placement requires a *contiguous* block of at least A_m bytes. Arm Cortex Microcontroller Software Interface Standard Neural Network (CMSIS-NN) kernels reduce inference cost on Arm Cortex-M processors [9]; MCUNet and MCUNetV2 reduce A_m via model/runtime co-design and patch inference [3], [4]; Machine Learning Performance (MLPerf) Tiny and MicroNets characterise accuracy and arena requirements across microcontroller-unit (MCU) hardware [2], [5]. These works optimise or report model-side memory use, but do not test final-state heap fragmentation under concurrent firmware stacks.

Deployment tooling and verification. TinyML deployment platforms such as Edge Impulse address fragmented embedded Machine Learning Operations (MLOps) workflows and support deployment across heterogeneous hardware targets [10]. Compiler frameworks such as the Tensor Virtual Machine (TVM) automate target-specific graph and operator optimisation [11]. These workflows help generate and benchmark deployable binaries, but their fit checks generally remain model/runtime estimates or target benchmarks rather than empirical probes of the largest contiguous internal heap block after application services are already active.

Heap profiling and fragmentation. The ESP-IDF capability-aware allocator [6], [7] exposes `heap_caps_get_largest_free_block()`, which reports the largest single free region per capability class. ESP-IDF heap-debugging facilities also provide heap

TABLE I
FIRMWARE AND WORKLOAD CONFIGURATION.

Item	Configuration
MCU / IDF	ESP32-S3, ESP-IDF v5.5.3, 240 MHz
PSRAM	8 MB Octal, 80 MHz, Error Correction Code (ECC) enabled; 32 KB reserved for Direct Memory Access (DMA)
Camera	OV2640 Joint Photographic Experts Group (JPEG) Quarter Video Graphics Array (QVGA), quality 12; one frame buffer in PSRAM
Local MQTT	Local Area Network (LAN) Mosquitto [14], Quality of Service (QoS) 1, 512 B payload, 5 Hz
Cloud MQTT	HiveMQ Cloud <code>mqtt://8883</code> , QoS 1, 512 B, 5 Hz
Wi-Fi default	Static receive (RX) 10, dynamic RX/transmit (TX) 32/32
Wi-Fi reduced	B1: static RX 6, RX/TX 16/16; B2: static RX 5, RX/TX 16/16
Latency test	<code>esp_person_detection</code> [15], 131,072 B arena

information, corruption detection, allocation hooks, and heap tracing for memory-leak and fragmentation analysis [7]. This value, not total free memory, governs arena placement. This paper combines that call with state sequencing, empirical allocation probes, and attribution logging, building on bounded-time allocation principles from real-time embedded systems such as Two-Level Segregated Fit (TLSF) [13].

III. METHODOLOGY

A. Platform and configuration

Experiments used two ESP32-S3 boards with 8 MB Octal PSRAM under ESP-IDF v5.5.3 [12]: Board 1 (B1) was a Seeed Studio XIAO ESP32S3 Sense with an OV2640 camera, Board 2 (B2) was a generic ESP32-S3 module with the same IDF version and PSRAM class. Table I summarises the firmware configuration.

B. Deployment-state verification workflow (Algorithm 1)

The verifier implements the following six-step procedure for each deployment state s .

Listing 1. Algorithm 1: Deployment-state tensor-arena verifier.

```

Input: state  $s$ , model set  $M$ , guard  $G$ , step  $D=1024$ 
Output: classification label for each  $m$  in  $M$ 

1. INIT( $s$ ):
   Initialise all peripherals and workloads
   for state  $s$  (Wi-Fi, camera, MQTT/TLS, ...).
   Record: free_int, largest_reported,
           free_psram, alloc_count, free_count.

2. PROBE  $T_{\text{empirical}}(s)$ :
    $\text{size} \leftarrow \text{largest\_reported}$ 
   while heap_caps_malloc(size, INTERNAL|8BIT) fails:
      $\text{size} \leftarrow \text{size} - D$ 
    $T_{\text{empirical}}(s) \leftarrow \text{size}$ 
   free all probe allocations.

3. GUARD:
    $T_{\text{safe}}(s) \leftarrow T_{\text{empirical}}(s) - G$ 

4. CLASSIFY each model  $m$  with arena  $A_m$ :
   if  $A_m \leq T_{\text{safe}}$ : INTERNAL_SAFE
   elif  $A_m \leq T_{\text{empirical}}$ : BORDERLINE
   else: PSRAM_REQUIRED

5. PLACE: allocate actual tensor arena for  $m$ ;
   verify via esp_ptr_external_ram().

6. REPORT: log classification, placement,
    $T_{\text{empirical}}$ ,  $T_{\text{safe}}$ , heap stats.

```

TABLE II
PAPER-FACING BENCHMARK MODES AND PRIMARY EMITTED RECORDS.

Mode	Purpose	Primary records
12	measure model arena use	ARENACSV, ALLOCATECSV
14	deployment-state verifier	MEMCSV, THRESHCSV, VERIFYCSV
19	repeated guard variance	GUARDVARCSV, GUARDVARSUMMARYCSV
24	Wi-Fi heap attribution	WIFIHEAPSNAPCSV, WIFIHEAPDELTACSV
25	forced placement latency	FORCEDPLACEMENTCSV, SUMMARYCSV

The guard $G = 16,384$ B is derived from twice the observed peak-to-peak MQTT-active threshold range (4,096 B over 10 samples), providing a conservative safety factor of $4\times$. Production deployments should re-derive G from at least 30 repeated measurements under their own network conditions. The step size $D = 1,024$ B controls probe resolution; smaller values refine the estimate at the cost of more allocations.

C. Benchmark modes and emitted records

The firmware emits machine-readable comma-separated-value (CSV)-like log records so that the paper tables can be regenerated without manually transcribing serial output. Table II summarises the benchmark modes used in this paper. The key design choice is to keep measurement roles separate: arena measurement determines A_m , deployment verification determines $T_{\text{empirical}}$, attribution explains which transition moved the boundary, and forced placement measures the latency consequence of using PSRAM.

For deployment states involving MQTT, the verifier waits for a successful connection and telemetry publish path before using the heap snapshot as a valid deployment-state measurement. This prevents a failed-network state from being mistaken for a lighter valid state. The cloud/TLS rows should therefore be interpreted as connected TLS MQTT measurements for the specified broker and certificate path, not as generic post-boot heap measurements.

D. Statistical reporting

Each threshold measurement is reported as mean $\pm$ std with min, max, and n . The latency benchmark reports mean of run means $\pm$ std of run means, with averaged within-run 95th-percentile latency (P95). The two quantities are kept distinct: std of run means captures run-to-run repeatability, and P95 captures within-run jitter.

E. Heap attribution

Attribution snapshots are taken before and after nine Wi-Fi initialisation stages: boot/task start, Non-Volatile Storage (NVS) `nvs_flash_init`, esp_netif_init, event-loop creation, Wi-Fi netif creation, `esp_wifi_init`, mode/cfg, `esp_wifi_start`, and connection/Internet Protocol (IP). Metrics per transition: Δ free internal, Δ largest block, traced allocation bytes. Reduced-buffer runs vary static RX and dynamic RX/TX counts to attribute the `esp_wifi_init()` collapse to configurable provisioning.

TABLE III
MODEL ARTIFACTS USED FOR PLACEMENT CLASSIFICATION.

Model	Task family	Arena	Role in frontier
ds_cnn_s	Keyword spotting	57.3 KB	TLS borderline
resnet_quant	Image classification	90.5 KB	MQTT borderline
vww2_50_INT8	Visual wake words	96.7 KB	MQTT borderline
kws_micro_s	Keyword spotting	107.8 KB	MQTT boundary failure
esp_person_det.	Person detection	122.6 KB	latency-control model
ds_cnn_m	Keyword spotting	169.7 KB	always PSRAM-required

TABLE IV
BOARD 1 LOCAL-STATE GUARD-VARIANCE SUMMARY ($n=10$).

State	Mean	Min	Max	std	Interpretation
wifi	143,360	143,360	143,360	0	deterministic
+camera	124,928	124,928	124,928	0	deterministic
+udp	120,832	120,832	120,832	0	deterministic
+mqtt	105,882	104,448	108,544	1,382	variable boundary

F. Forced-placement latency

Arena placement is forced via capability flags:

```
// Internal SRAM
heap_caps_malloc(sz, MALLOC_CAP_INTERNAL|MALLOC_CAP_8BIT);
// PSRAM
heap_caps_malloc(sz, MALLOC_CAP_SPIRAM|MALLOC_CAP_8BIT);
```

Only `interpreter.Invoke()` is timed; setup, capture, and publish are excluded. Model, firmware state, and arena size are held constant across placement conditions.

G. Model set and accuracy scope

Table III lists the model artifacts, including 8-bit integer (INT8) models, used in the placement study. The set is chosen to span the internal-safe, borderline, and PSRAM-required regions of the measured frontier. It is not a statistical sample of TinyML workloads. The reported arena values are measured by the firmware harness and task accuracy is not re-evaluated in this paper. Any benchmark family names are used only to identify model provenance.

IV. RESULTS

A. Internal allocation thresholds

Table V reports empirical thresholds with full statistics. The first three states showed zero variance across 10 samples on both boards. The MQTT-active state varied by 4,096 B (range) on Board 1. Both boards reproduced the same monotone decline from Wi-Fi-only to MQTT-active; the largest cross-board difference (2,048 B) appears in the MQTT-active state.

Table IV summarises the guard-margin evidence used in the main classification. The zero entries are not missing data: those states returned the same empirical limit on every repeated probe. The only local state with observable short-run variation was MQTT-active operation. Using a 16 KB guard is therefore conservative for the measured local MQTT variance, but it is still a policy choice rather than a universal constant.

TABLE V
EMPIRICAL INTERNAL ALLOCATION THRESHOLDS ($n=10$ PER CELL EXCEPT TLS STATES; SAFE BUDGET = $T_{\text{emp}} - 16$ KB). STATES WITH ZERO VARIANCE SHOW A SINGLE VALUE.

State	Board 1 ($n = 10$)			Board 2 ($n = 10$)		
	Mean	std	Safe	Mean	std	Safe
wifi_connected	143,360	0	126,976	143,360	0	126,976
+camera	124,928	0	108,544	124,928	0	108,544
+udp	120,832	0	104,448	118,784	0	102,400
+mqtt	105,882	1,382	88,064–92,160	103,424	1,024	86,016–88,064
+tls ($n=100$)	63,427	351	47,104	64,922	1,079	49,152

All values in bytes. Safe budget uses T_{min} for variable states.

TABLE VI
CLOUD/TLS MQTT REPEATABILITY SUMMARY FOR THE
WIFI_CAMERA_MQTT_TLS_ACTIVE STATE.

Experiment	Board	n	Mean	Min–Max	std	MQTT result
within-session	B1	100	63,427	61,440–63,488	351	261/261
within-session	B2	100	64,922	61,440–65,536	1,079	260/260
cold-start boot	B1	26	64,709	63,488–65,536	820	78/78

B. Cloud/TLS MQTT state

The `wifi_camera_mqtt_tls_active` state used a HiveMQ Cloud `mqtt:// broker` (port 8883). Board 1 completed 261/261 and Board 2 completed 260/260 QoS 1 publishes across 100-repeat runs (Table V). A separate 26-boot cold-start experiment on Board 1 established a fresh TLS connection on every boot. Table VI summarises the repeated TLS evidence.

The 26-boot cold-start experiment confirms cross-reset reproducibility, but it is not a continuous-session reconnect test. Compared with local MQTT limits of 102,400–104,448 B, the cloud/TLS state reduced the boundary by 37–43 KB, reclassifying all models larger than 57 KB as PSRAM-REQUIRED. These values are specific to one broker/certificate configuration and should not be treated as general cloud-MQTT bounds.

C. Model placement classifications

Table VII shows placement across the deployment ladder. The key finding is that *boundary models change class* as realistic components are enabled. `esp_person_detection` moves from internal-safe under Wi-Fi alone to PSRAM-required under User Datagram Protocol (UDP) or MQTT, `kws_micronet_s` crosses from safe to PSRAM-required at the MQTT step. Large models are always PSRAM-required and are omitted for space.

D. Boundary-model sensitivity

The guard margin separates allocation success from engineering headroom. Under the 10 Board 1 MQTT-active samples, `resnet_quant` and `vww2_50_INT8` never exceeded the empirical allocation limit, but both lose the 16 KB guard and are therefore reported as borderline. By contrast, `kws_micro_s` exceeded the empirical limit in 9/10 samples, and `esp_person_detection` exceeded it in 10/10 samples. Table VIII summarises the boundary cases. This helps

TABLE VII

MODEL PLACEMENT ACROSS THE DEPLOYMENT LADDER (BOTH BOARDS).

Model	A_m	Wi-Fi	+Cam	+UDP	+MQTT
ds_cnn_s	57.3 KB	Safe	Safe	Safe	Safe
resnet_quant	90.5 KB	Safe	Safe	Safe	Border.
vww2_50_INT8	96.7 KB	Safe	Safe	Safe	Border.
kws_micro_s	107.8 KB	Safe	Safe	Border.	PSRAM
esp_person_det.	122.6 KB	Safe	Border.	PSRAM	PSRAM

TABLE VIII

BOUNDARY-MODEL SENSITIVITY UNDER BOARD 1 MQTT-ACTIVE SAMPLES ($n=10$) AND CLOUD/TLS LIMITS.

Model	Arena	MQTT failures	Local MQTT label	TLS label
resnet_quant	90.5 KB	0/10	Borderline	PSRAM
vww2_50_INT8	96.7 KB	0/10	Borderline	PSRAM
kws_micro_s	107.8 KB	9/10	PSRAM	PSRAM
esp_person_det.	122.6 KB	10/10	PSRAM	PSRAM

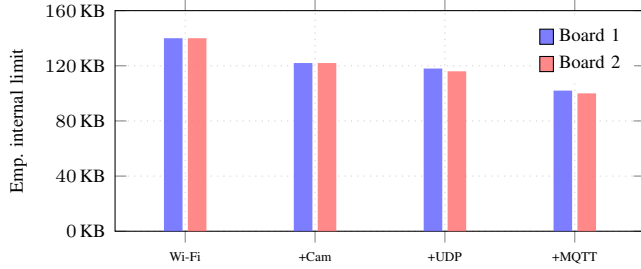


Fig. 1. Empirical contiguous internal limit across the deployment ladder (rounded to nearest KB). The MQTT-active state is the tightest tested configuration.

interpret the classification labels: *borderline* means an allocation can fit in the observed state but lacks guard headroom; *PSRAM-required* means the requested arena is larger than the empirical contiguous internal limit.

Total free vs. contiguous limit. In the Board 2 MQTT-active state, 177,099 B total free internal memory is available while the empirical contiguous limit is only 102,400 B. A model requiring 122,580 B appears feasible from total-free alone but fails the contiguous test. Fig. 1 illustrates this across the ladder.

This is the deployment failure mode the verifier is designed to catch. The tensor arena is not a collection of small independent allocations that can be scattered over the heap; it is a single region handed to the runtime. Once the largest internal block falls below the arena need, the system must either fail allocation, select a smaller model, tune the firmware stack, or place the arena in PSRAM and accept the latency cost.

V. HEAP ATTRIBUTION

`esp_wifi_init()` is the dominant structural transition for contiguous internal memory. Earlier stages (NVS, event-loop, netif creation) consume internal memory but do not reduce the largest block. Fig. 2 shows the Board 1 waterfall.

On Board 1, the default `esp_wifi_init()` produced a 32,768 B largest-block collapse from 45 traced inter-

TABLE IX

BOARD 2 `ESP_WIFI_INIT()` COMPARISON.

Config	RX/TX profile	Largest after	Block drop	Net traced
Default	10, 32/32	131,072 B	32,768 B	33,160 B
Reduced	5, 16/16	139,264 B	24,576 B	25,140 B

TABLE X

BOARD 1 DEFAULT VS. REDUCED-BUFFER `ESP_WIFI_INIT()` COMPARISON.

Metric	Default	Reduced	Δ
Largest-block drop	32,768 B	24,576 B	-8,192 B
Traced alloc bytes	34,268 B	27,852 B	-6,416 B
Largest block after init	131,072 B	139,264 B	+8,192 B

nal allocations totalling 34,268 B (net 33,160 B). Contributors include the Wi-Fi driver task stack (6,656 B), repeated `wDev_Rxbuf_Init` allocations ($\approx 1,604$ B each), adapter state, physical-layer (PHY) init, and Wi-Fi Protected Access (WPA)/supplicant state. Reducing static RX buffers from 10 to 6 and dynamic RX/TX from 32/32 to 16/16 recovered 8,192 B (Table X), corresponding exactly to four removed RX buffers ($4 \times 1,604 = 6,416$ B traced, 8 KB largest-block gain):

$$\frac{8,192}{32,768} \approx 25\% \text{ of the default collapse explained.} \quad (1)$$

Board 2 reproduced the same pattern: 32,768 B collapse under default settings, 8,192 B recovery under reduced buffering (static RX 10 $\rightarrow$ 5, RX/TX 32/32 $\rightarrow$ 16/16), with identical pre-`esp_wifi_init()` heap state on both configurations (248,203 B free, 163,840 B largest), ruling out a pre-existing shape difference.

The corrected Board 2 reduced-buffer run is important because it removes the ambiguity present in an earlier build: the firmware itself printed `WIFIATTRIB_CONFIGCSV,reduced_rx_tx,5,16,16`, confirming that the intended Wi-Fi profile was active. After `esp_wifi_init()`, the default run reported a 131,072 B largest block and the reduced run reported 139,264 B. Net traced internal allocation also fell from 33,160 B to 25,140 B, an 8,020 B reduction matching five removed 1,604 B static RX buffers. This independent reproduction strengthens the interpretation that configurable Wi-Fi provisioning is a real placement lever, while the exact threshold remains build-specific.

VI. PLACEMENT-CONTROLLED LATENCY

`esp_person_detection` was benchmarked in the `wifi_connected` state with a 131,072 B arena ($n=10$ runs, 100 warm invocations per run). Table XI reports the mean of run means and std of run means; P95 is the average per-run 95th-percentile invocation latency. The PSRAM penalty was 34.6% on Board 1 and 34.7% on Board 2. The tight std of run means (≤ 0.06 ms) confirms high run-to-run repeatability,

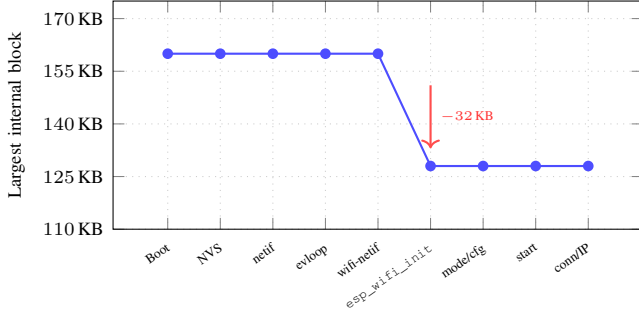


Fig. 2. Board 1 Wi-Fi attribution waterfall (default buffers). The 32 KB largest-block drop occurs entirely at `esp_wifi_init()`, before camera or MQTT initialisation.

TABLE XI
FORCED-PLACEMENT INFERENCE LATENCY ($n=10$ RUNS, WIFI_CONNECTED STATE). STD = STD OF RUN MEANS.

Board	Placement	Mean (ms)	std (ms)	P95 (ms)	Penalty
B1	Internal	59.558	0.025	60.970	—
B1	PSRAM	80.185	0.060	81.715	+34.6%
B2	Internal	59.725	0.040	60.644	—
B2	PSRAM	80.452	0.048	81.510	+34.7%

latency variation is captured by P95. Placement classification labels therefore map directly to material latency differences.

VII. REPRODUCIBILITY

All source code, build configuration, model files, heap logs, and analysis scripts are provided in the accompanying repository at <https://github.com/aarong-uol/esp32s3-tinyml-verifier>. The repository is organised as follows:

- `firmware/` — ESP-IDF component containing the verifier, attribution harness, and forced-placement benchmark. Build with `idf.py build flash monitor`; default target is ESP32-S3.
- `sdkconfig.defaults` and `sdkconfig.diff` canonical `sdkconfig` for both default and reduced-buffer Wi-Fi configurations. Key differences: `ESP_WIFI_STATIC_RX_BUFFER_NUM`, `ESP_WIFI_DYNAMIC_RX_BUFFER_NUM`, `ESP_WIFI_DYNAMIC_TX_BUFFER_NUM`.
- `models/` — Quantised `.tflite` files for all six models in Table III. Model sources and benchmark family provenance are documented in `models/SOURCES.md`.
- `logs/` — Raw heap-log CSV files for all deployment states, both boards, default and reduced-buffer runs, and TLS guard-variance runs. Filenames encode board, state, and buffer configuration (e.g., `b1_mqtt_tls_default_n100.csv`).
- `analysis/` — Python scripts to reproduce Tables V–XI and Figs. 1–2 from the raw logs. Run `python analysis/summarise.py logs/` to regenerate all tables.

To reproduce the threshold measurements, flash the firmware with the target `sdkconfig`, set the desired deployment state via `idf.py menuconfig`

(component: `VERIFIER_STATE`), and capture the serial log. The Python scripts parse the `VERIFYCSV` and `WIFIATTRIB_CONFIGCSV` log markers to extract heap metrics.

The artifact also records the exact CSV schemas emitted by each mode: `MEMCSV` for heap snapshots, `THRESHCSV` for empirical limits, `VERIFYCSV` for model classifications, `WIFIHEAPSNAPCSV` and `WIFIHEAPDELTACSV` for attribution, and `FORCEDPLACEMENTCSV` for latency validation. This keeps the paper tables regenerable from raw serial logs rather than manually copied measurements.

VIII. DISCUSSION

Placement is a deployment-state property. The main lesson is that tensor-arena feasibility should be treated as a property of the complete firmware state, not of the model alone. The verifier exposes three different boundaries: the model arena requirement, the reported largest free internal block, and the empirically allocatable internal block after all deployment services are running. The last of these is the most deployment-relevant value. In the tested stack, total free internal memory remained larger than several model arenas, but the contiguous internal allocation limit had already fallen enough to force borderline or PSRAM placement.

Attribution turns the result into an engineering action. Without attribution, a failed internal allocation only shows that the final heap is constrained. The Wi-Fi waterfall shows where the constraint first appears: `esp_wifi_init()` establishes the heap shape before camera or MQTT activity begins. The reduced-buffer runs then show that part of the collapse is configurable, recovering 8 KB of contiguous internal memory on both corrected board runs. This does not make buffer tuning a universal fix, but it gives a concrete optimisation loop: verify the final state, identify the dominant heap transition, tune only the relevant subsystem, and rerun the verifier.

Borderline models need reliability evidence. The borderline label is intentionally stricter than a one-time successful allocation. A model may allocate internally in a clean run but lose guard headroom under different access point (AP) behaviour, Received Signal Strength Indicator (RSSI), publish timing, or firmware build order. For this reason the paper reports repeated threshold measurements and uses a 16 KB guard. A production deployment should replace this fixed guard with a placement-success estimate across the network conditions it will actually encounter.

TLS result interpretation. The cloud/TLS MQTT state is useful because it changes placement decisions, reducing the internal boundary by roughly 37–43 KB relative to local MQTT. However, the current TLS result is end-to-end: it measures the final pressure of Wi-Fi + camera + TLS MQTT, but does not byte-attribute certificate parsing, mbedTLS buffers, MQTT client state, Transmission Control Protocol/Internet Protocol (TCP/IP) buffers, or reconnect paths. A dedicated TLS attribution mode is future work.

Limitations. This is a two-board ESP32-S3 / ESP-IDF v5.5.3 case study, so exact byte thresholds should not be generalised across boards, IDF versions, Wi-Fi settings, brokers, certificate chains, or application firmware. Task accuracy was not

evaluated; accuracy values are literature or benchmark-family anchors only. The reset-to-reset TLS experiment tests cold-start repeatability, not continuous-session reconnect stress.

IX. CONCLUSION

This paper presented a deployment-state tensor-arena verification workflow for TinyML on ESP32-S3, comprising empirical contiguous-block probing, Wi-Fi transition attribution, and forced-placement latency validation. The empirical internal limit fell from 143,360 B under Wi-Fi-only ($\text{std}=0$, $n=10$) to 102,400–104,448 B under Wi-Fi+camera+MQTT ($\text{std}\leq 1,382$ B), and to 61,440–65,536 B under cloud/TLS MQTT ($n=100$). `esp_wifi_init()` caused a 32 KB largest-block collapse; reduced buffer provisioning recovered 8 KB. PSRAM placement increased inference latency by 34.6–34.7% ($n=10$ runs). The practical takeaway is to run this workflow probe the final deployment state, apply a guard margin, verify actual placement, and re-verify after any stack change rather than inferring placement from model size or total free memory alone.

REFERENCES

- [1] R. David, J. Duke, A. Jain, V. J. Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, S. Regev, R. Rhodes, T. Wang, and P. Warden, “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems,” in *Proc. 4th MLSys Conf.*, 2021, arXiv:2010.08678.
- [2] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau, U. Oguntebi, G. Enderby, L. Mascarenas, P. Warden, and T. Whatmough, “MLPerf Tiny Benchmark,” in *Proc. 35th Conf. Neural Inf. Process. Syst. (NeurIPS) Datasets and Benchmarks Track*, 2021, arXiv:2106.07597.
- [3] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, “MCUNet: Tiny Deep Learning on IoT Devices,” in *Proc. 34th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2020, arXiv:2007.10319.
- [4] J. Lin, W.-M. Chen, H. Cai, C. Gan, and S. Han, “MCUNetV2: Memory-Efficient Patch-Based Inference for Tiny Deep Learning,” in *Proc. 35th Conf. Neural Inf. Process. Syst. (NeurIPS)*, 2021, arXiv:2110.15352.
- [5] C. Banbury, C. Zhou, I. Fedorov, R. Matas, U. Thakker, D. Gope, V. J. Reddi, M. Mattina, and P. N. Whatmough, “MicroNets: Neural Network Architectures for Deploying TinyML Applications on Commodity Microcontrollers,” in *Proc. 4th MLSys Conf.*, 2021, arXiv:2010.11267.
- [6] Espressif Systems, “Heap Memory Allocation,” in *ESP-IDF Programming Guide v5.5*, 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32s3/api-reference/system/mem_alloc.html
- [7] Espressif Systems, “Heap Memory Debugging,” in *ESP-IDF Programming Guide v5.5*, 2024. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v5.5/esp32s3/api-reference/system/heap_debug.html
- [8] Espressif Systems, “Support for External RAM,” in *ESP-IDF Programming Guide*, ESP32-S3, 2026. [Online]. Available: <https://docs.espressif.com/projects/esp-idf/en/stable/esp32s3/api-guides/external-ram.html>
- [9] L. Lai, N. Suda, and V. Chandra, “CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs,” arXiv:1801.06601, 2018.
- [10] S. Hymel, C. Banbury, D. Situnayake, A. Elum, C. Ward, M. Kelcey, M. Baaijens, M. Majchrzycki, J. Plunkett, D. Tischler, A. Grande, L. Moreau, D. Maslov, A. Beavis, J. Jongboom, and V. J. Reddi, “Edge Impulse: An MLOps Platform for Tiny Machine Learning,” arXiv:2212.03332, 2022.
- [11] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy, “TVM: An Automated End-to-End Optimizing Compiler for Deep Learning,” arXiv:1802.04799, 2018.
- [12] Espressif Systems, *ESP32-S3 Technical Reference Manual*, v1.4, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32-s3_technical_reference_manual_en.pdf
- [13] M. Masmano, I. Ripoll, A. Crespo, and J. Real, “TLSF: A New Dynamic Memory Allocator for Real-Time Systems,” in *Proc. 16th Euromicro Conf. Real-Time Syst. (ECRTS)*, Catania, Italy, Jun. 2004, pp. 79–88, doi: 10.1109/ECRTS.2004.37.
- [14] Eclipse Foundation, “Eclipse Mosquitto: An Open Source MQTT Broker,” version 2.0, 2023. [Online]. Available: <https://mosquitto.org/documentation/>
- [15] Espressif Systems, “TensorFlow Lite Micro person_detection Example for ESP32,” ESP-IDF Examples, 2023. [Online]. Available: https://github.com/espressif/esp-idf/tree/master/examples/peripherals/camera/camera_dsp